# Fundamentals of Machine Learning
# Final Hard Mock Midterm: Solutions

**Every proof in a few lines, every number checked, every judgement with its mechanism and its trap**

Fundamentals of Machine Learning

2026-10-31

## Part A. True or false

**A1. False.** Zero training error only shows the model memorised the sample. With continuous features no two flows coincide almost surely, so even pure label noise can be fitted perfectly. The Bayes error is a property of $p(y \mid x)$, not of a fitted sample.

**A2. True.** With $(x, x')$ the Bayes rule can always ignore $x'$ and reproduce the best rule on $x$. The best rule on the larger representation is at least as good.

**A3. True.** Least squares minimises the training MSE over *all* weight vectors. Ridge picks one particular vector, so its training MSE is at least as large. Ridge buys validation error with training error.

**A4. True.** A strictly increasing map keeps the ranking, so the top 100 are the same flows (ties aside). Log loss depends on the probability values themselves, so it can differ.

**A5. False.** $M$ only shrinks the $(1-\rho)\sigma^2/M$ term. The correlation $\rho$ comes from the bootstrap and $m_{\text{try}}$, and the floor $\rho\sigma^2$ does not move.

**A6. True.** The squared-error split criterion is unchanged when a constant is subtracted from the targets, so the tree fitted to $y - \bar{y}$ has the same splits. Each leaf predicts $\bar{y} + \overline{(y-\bar{y})}_L = \bar{y}_L$, exactly the tree fitted to $y$.

**A7. False.** Zero means *this fitted model* does not rely on it. The feature can still be informative but redundant given a correlated feature (masking), or useful only jointly (C2), or simply unused by the model.

**A8. False.** The threshold depends only on the costs. Once the posterior refers to the deployment prior, apply the same $C_{FP}/(C_{FP}+C_{FN})$. Correct the posterior *or* move the threshold; doing both corrects twice.

**A9. True.** $\log p(x \mid c) = -\sum_j (x_j - \mu_{cj})^2/(2\sigma_j^2) + \text{const}$. With $\sigma_j$ the same in both classes, the $x_j^2$ terms cancel in the log-odds, which is linear in $x$.

**A10. False.** Gibbs gives $H(p,q) \geq H(p)$, not $H(q)$. Counterexample: $p = (1,0)$, $q = (0.9, 0.1)$: $H(p,q) = -\log_2 0.9 = 0.152$ bits $< H(q) = 0.469$ bits.

## Part B. Multiple choice

**B1. (a), (b), (d).** The gap 0.01/0.30 is variance. More data and a minimum leaf size reduce it, and bagging averages unstable trees. Interactions (c) and leaves of size 1 (e) *add* capacity.

**B2.** **(b)** $1-(1-1/10)^{10} = 1-0.349 = 0.651$. 0.632 is the limit $1-e^{-1}$ as $N \to \infty$, not the value at $N = 10$.

**B3.** **(a), (c).** $e^{s_c+a}/\sum_k e^{s_k+a} = e^{s_c}/\sum_k e^{s_k}$; (c) is the case $a = -\max s$ (the numerically stable form). Multiplying by 2 sharpens the distribution; changing one score changes it.

**B4.** **(b) only.** Expected cross-entropy $-0.7\ln q - 0.3\ln(1-q)$: $q = 0.5$: 0.693; 0.6: 0.632; 0.4: 0.795; 0.9: 0.765; 0.99: 1.389 nats. Being overconfident in the right direction (0.9) is worse than hedging, as in Chapter 1's example.

**B5.** **(a), (c), (d).** Raising the threshold removes alerts, true and false, so the counts of TP, FP and alerts can only fall: recall and FPR cannot increase. Precision can go either way, because the removed alerts may be mostly true positives.

**B6.** **(a)** Parent $1-0.4^2-0.6^2 = 0.48$; left $1-0.75^2-0.25^2 = 0.375$; right $1-(1/6)^2-(5/6)^2 = 0.278$; decrease $0.48 - 0.4\times 0.375 - 0.6\times 0.278 = 0.163$.

**B7.** **(a), (b).** Gain $\frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma$. The learning rate is applied *after* the tree is built. The number of rounds controls how many trees are built, not whether a split is kept.

**B8.** **(a), (d).** (b) is the future; (c) is information published after the decision. Both are leakage, even though (c) is "about" the same source.

## C1. Three penalties on one coefficient

**(a)** If $w \neq 0$ the objective is $\frac{1}{2}(w-z)^2 + \lambda$, minimised at $w = z$ with value $\lambda$. If $w = 0$ it is $\frac{1}{2}z^2$. Keep $w = z$ iff $\lambda < z^2/2$, i.e. $|z| > \sqrt{2\lambda}$; otherwise $\hat{w} = 0$. Kept coefficients are *not shrunk at all.*
**(b)** For $w > 0$: the derivative $w - z + \lambda_1 + 2\lambda_2 w = 0$ gives $w = (z-\lambda_1)/(1+2\lambda_2)$, admissible iff $z > \lambda_1$. For $w < 0$: $w = (z+\lambda_1)/(1+2\lambda_2)$, admissible iff $z < -\lambda_1$. If $|z| \leq \lambda_1$ neither is admissible and the minimum is at the kink $w = 0$. Together: soft thresholding, then ridge division.
**(c)**

| | | |
|---|---|---|
| best subset, $\lambda = 0.5$ (cut at $\sqrt{1} = 1$) | $(3,\ 0,\ 0,\ -2)$ | big ones kept, small ones zeroed |
| lasso, $\lambda = 1$ | $(2,\ 0,\ 0,\ -1)$ | zeros, big ones shifted by 1 |
| elastic net, $\lambda_1 = 1, \lambda_2 = 0.5$ | $(1,\ 0,\ 0,\ -0.5)$ | zeros, then halved |
| ridge, $\lambda = 0.5$ | $(1.5,\ 0.4,\ -0.2,\ -1)$ | no zeros, everything halved |

Only best subset leaves the large coefficients untouched. Lasso biases even the strongest features by $\lambda$; ridge and the elastic net shrink proportionally.
**(d)** The fit depends only on $c = w_1 + w_2$. Lasso: $\lambda_1(|w_1| + |w_2|) \geq \lambda_1|c|$, with equality for *every* split of the same sign. So all such splits give the same objective, the minimiser is not unique, and the solver's choice is arbitrary. Elastic net: add $\lambda_2(w_1^2 + w_2^2) = \lambda_2\left[\frac{1}{2}c^2 + \frac{1}{2}(w_1 - w_2)^2\right]$, which is smallest at $w_1 = w_2 = c/2$, unique. *Lesson:* lasso on (nearly) duplicated features picks one arbitrarily, and the choice can flip between samples, so a zero coefficient does not mean "irrelevant". The $L_2$ part groups correlated features.
**(e)** (i) Lasso (or best subset): sparse and readable; if the list must be stable under correlated features, the elastic net. (ii) Ridge (or the elastic net with a large $L_2$ part): many small, shared effects, where sparsity would discard signal. Best subset is combinatorial because, with correlated features, whether to include one depends on which others are included: $2^d$ subsets. With orthogonal features the objective separates into $d$ independent one-coefficient problems, so each keep-or-drop decision is made alone.

**Course:** Essential ch.3 "Controlling effective capacity" (ridge/lasso one-variable box), "Selecting features" (subset selection is combinatorial); review Q12, Q14.

**Tempting wrong answer.** "Lasso is best subset with a smooth penalty." The zeros match, but lasso shrinks every kept coefficient by $\lambda$ while best subset keeps it intact.

## C2. A feature that is the same in both classes

**(a)** The second coordinate of both means is 0 and its variance is $\Sigma_{22} = 1$, so $x_2 \mid y \sim N(0,1)$ for both classes: $x_2$ is independent of $y$, $\mathrm{Cov}(x_2, Y) = 0$, and on its own it gives error 0.5. A correlation filter, or any one-feature-at-a-time test, drops it.
**(b)** $\Delta = (\delta, 0)^\top$, so $w = \frac{1}{1-\rho^2}(\delta,\ -\rho\delta)^\top = (2.78,\ -2.22)^\top$ and $w^\top x = \frac{\delta}{1-\rho^2}(x_1 - \rho x_2)$. Server 2 measures the congestion. $\rho x_2$ is the part of $x_1$ that congestion explains, and subtracting it removes the shared noise: $\mathrm{Var}(x_1 - \rho x_2 \mid y) = 1 - \rho^2 = 0.36$, while the attack shift $\delta$ is untouched. The model uses server 2 as a *control*: a negative weight on a feature unrelated to the label.
**(c)** $D^2 = \Delta^\top \Sigma^{-1} \Delta = \delta^2 \cdot \frac{1}{1-\rho^2} = 2.78$, so $D = 1.67$. Error $\Phi(-0.83) = 0.202$ with both features, against $\Phi(-0.5) = 0.309$ with $x_1$ alone ($D = \delta = 1$). A feature that is "useless" on its own cuts the error by a third.
**(d)** Naive Bayes multiplies $p(x_1 \mid y)\, p(x_2 \mid y)$. The $x_2$ factor is the same for both classes, so its likelihood ratio is 1 and it is ignored: error 0.309, as with $x_1$ alone. In Tutorial 1, correlated features *repeat* the same evidence and naive Bayes counts it twice: overconfidence, a calibration problem. Here the correlation *is* the evidence (noise cancellation), and the independence assumption throws it away: lost accuracy.
**(e)** At the root, any split on $x_2$ leaves both children at the class proportions of the parent ($x_2$ is independent of $y$), so its impurity decrease is about 0. The greedy choice is $x_1$. $x_2$ only becomes informative *conditionally* on $x_1$. The ideal boundary $x_1 - 0.8x_2 = c$ is oblique, so an axis-aligned tree needs a deep staircase of splits, which means more data and more variance. Engineering the feature $x_1 - \rho x_2$ (a representation change) gives it to a tree in one split.
**(f)** At deployment $\Sigma = I$. The score's class means differ by $w^\top \Delta = \delta^2/(1-\rho^2)$, and its variance is $\|w\|^2 = \delta^2(1+\rho^2)/(1-\rho^2)^2$. The ratio is $\delta/\sqrt{1+\rho^2} = 1/\sqrt{1.64} = 0.78$. The means are unchanged, so the equal-prior threshold is still the midpoint, and the error is $\Phi(-0.39) = 0.348$: worse than $x_1$ alone (0.309) and far from the validated 0.202. *Lesson:* a gain that comes from a *relation between features* disappears when the relation breaks, and the negative weight then injects noise. Validate across environments, monitor input correlations, and retrain when they move.

**Course:** Essential ch.4 LDA (Mahalanobis, shared covariance), naive Bayes; ch.3 feature selection by correlation filtering; ch.5 greedy splits; T1 7.

**Tempting wrong answer.** "$x_2$ has the same distribution in both classes, so it cannot help." The marginal is uninformative; the joint distribution is not.

## C3. Bagging the nearest neighbour

**(a)** Each of the $N$ independent draws picks one of $N$ points uniformly, so it avoids $j$ given points with probability $1 - j/N$. All $N$ draws avoid them with probability $(1 - j/N)^N \approx e^{-j}$.
**(b)** $x_{(i)}$ is the nearest point present iff $x_{(1)}, \ldots, x_{(i-1)}$ are all absent and $x_{(i)}$ is not. So $w_i = (1 - \frac{i-1}{N})^N - (1 - \frac{i}{N})^N \approx e^{-(i-1)} - e^{-i} = (1 - e^{-1})e^{-(i-1)}$. Values: $w_1 = 0.632$ (the familiar in-bag probability), $w_2 = 0.233$, $w_3 = 0.086$, $w_4 = 0.031$. Sum: $(1-e^{-1}) \sum_{i \geq 0} e^{-i} = (1-e^{-1})/(1-e^{-1}) = 1$.
**(c)** The noises are independent, so $\mathrm{Var} = \sigma^2 \sum_i w_i^2 = \sigma^2 (1 - e^{-1})^2 \sum_{i \geq 0} e^{-2i} = \sigma^2 \frac{(1-e^{-1})^2}{1-e^{-2}} = \sigma^2 \frac{1-e^{-1}}{1+e^{-1}} = 0.462\, \sigma^2$. Setting $\sigma^2/k = 0.462\, \sigma^2$ gives $k \approx 2.2$: bagged 1-NN behaves like a 2-NN whose weights decay geometrically. Bagging more than halved the variance.
**(d)** The bias is $\sum_i w_i f(x_{(i)}) - f(x_0)$: 37% of the weight now sits on points farther than the nearest one, where $f$ differs more from $f(x_0)$. The bias therefore grows, roughly to the level of a 2-NN. Bagging here is a bias–variance trade, the same direction as increasing $k$ slightly.
**(e)** Bagging helps any *unstable* predictor, and 1-NN is unstable: its prediction is a single noisy response. (c) shows a 54% cut in variance. A 50-NN prediction already averages 50 responses (variance about $\sigma^2/50$). Removing about 37% of the points only replaces some neighbours

by slightly farther ones, so the average barely moves: little variance to remove, and a little extra bias. Stable learners gain almost nothing from bagging (same as logistic regression in the Understanding Mock).

**Course:** Essential ch.5 bagging ($1 - e^{-1}$, variance of an average), ch.3 bias–variance and $k$-NN capacity.

**Tempting wrong answer.** "Bagged 1-NN is still 1-NN, so nothing changes." Each bag is 1-NN, but the average is a weighted neighbourhood.

## C4. Two ways to shrink a boosted tree

**(a)** By induction $F_{m-1}$ is constant on $L$ (true for $F_0$). The residuals on $L$ are $y_i - F_{m-1}$, and the tree's leaf value is their mean, $\bar{y}_L - F_{m-1}$. So $F_m = F_{m-1} + \eta(\bar{y}_L - F_{m-1})$, i.e. $\bar{y}_L - F_m = (1-\eta)(\bar{y}_L - F_{m-1})$. Iterating gives $\bar{y}_L - F_m = (1-\eta)^m(\bar{y}_L - F_0)$.
**(b)** $\eta = 0.1$: $F_{10} = 5 - 4(0.349) = 3.61$, $F_{50} = 5 - 4(0.0052) = 4.98$. $\eta = 0.3$: $F_{10} = 5 - 4(0.028) = 4.89$. The remaining gap is about $e^{-\eta m}$, so progress depends on $\eta m$: dividing $\eta$ by 3 needs about 3 times as many rounds for the same fit, which is why validation selects more rounds at a small $\eta$.
**(c)** $g_i = \partial \ell / \partial F = F_i - y_i$, $h_i = 1$. So $G = -\sum_{i \in L}(y_i - F_i)$, $H = n_L$, and $w^* = -G/(H + \lambda) = \sum_{i \in L}(y_i - F_i)/(n_L + \lambda) = \frac{n_L}{n_L + \lambda}\,\overline{(y - F)}_L$.
**(d)** $\eta$ applies the *same* factor to every leaf. $\lambda = 10$ keeps $2/12 = 17\%$ of the mean residual of a 2-point leaf, but $200/210 = 95\%$ for a 200-point leaf. A mean of two noisy residuals has high variance ($\sigma^2/2$), so $\lambda$ shrinks it hard toward "no correction". A mean of 200 is reliable and is left almost intact. $\lambda$ is the bias–variance device aimed at small leaves. $\eta$ slows everything uniformly and leaves room for later trees (T2 4c), without knowing which leaves are noisy.
**(e)** Toward $F_0 = \bar{y}$: $F_m - F_0 = [1 - (1-\eta)^m](\bar{y}_L - F_0)$. Each leaf mean is shrunk toward the global mean by the factor $1 - (1-\eta)^m$, so stopping early is like ridge on the leaf values. This is early stopping as implicit regularisation (review Q13).

**Course:** Essential ch.5 "Gradient boosting for regression", "XGBoost objective"; ch.3 early stopping; T2 4, 5, 7d.

**Tempting wrong answer.** For (d): "$\eta$ and $\lambda$ do the same thing". $\lambda$ depends on the leaf size; $\eta$ does not.

## C5. How many attacks do we have?

**(a)** $\mathbb{E}_x[\mathbb{P}(Y = 1 \mid x)] = \int p(x)\,\mathbb{P}(Y = 1 \mid x)\,dx = \int p(x, Y = 1)\,dx = \mathbb{P}(Y = 1)$ (law of total probability). Check: over any large window drawn from the training distribution, a calibrated model's average output must match the base rate. A drift of the average signals a shift or a calibration problem.
**(b)** Not necessarily. A perfect separator outputs 1 on attacks and 0 on benign flows, so its average is exactly $\pi_d$. An uninformative model outputs $\pi_t = 0.5$ everywhere, so its average is 0.5 whatever $\pi_d$ is. Real models lie in between: the raw average is pulled toward $\pi_t$. Since $0.37 < 0.5$, the true $\pi_d$ is probably below 0.37.
**(c)** (a) holds at deployment for the *deployment* posterior, so $\pi_d = \mathbb{E}_d[r_d(x)]$, estimated by the sample mean. But $r_d$ itself depends on $\pi_d$: a fixed-point equation. One update from 0.37 (ratios $0.37/0.5 = 0.74$ and $0.63/0.5 = 1.26$): $r_d = (0.841,\ 0.468,\ 0.128,\ 0.061,\ 0.030)$, mean **0.306**. The next updates give 0.273, and the iteration converges to about 0.23. This is the EM estimate of a new prior.
**(d)** The shift is not a prior shift: $p(x \mid y)$ changed (new attack types, new services), so the correction formula is wrong. Or the model was never calibrated ($r_t$ overconfident, e.g. boosting or naive Bayes), so (a) did not hold even in training.
**(e)** Non-attack causes: a new benign service that looks like attacks (IT's vulnerability scanner, a backup job), i.e. covariate shift; a pipeline change (bytes logged in KB, a field missing and replaced by a default value). To tell them apart: monitor the input distributions (ranges, missing rates,

unseen categories, low $p(x)$ under a generative model); see whether the extra score mass comes from one host or service; check whether the new score histogram is a re-weighting of the old class-conditional histograms (prior shift) or has a new shape; hand-label a sample of the new high-score flows.

**Course:** Essential ch.2 Bayes rule and total probability; ch.4 "Prior shift"; T1 6e; Understanding Mock 4c.

**Tempting wrong answer.** For (b): "the average predicted probability is the attack rate". Only for a calibrated model on its own distribution.

## C6. The post-mortem

**(a) Expert collapse.** The gate and the experts are trained jointly. An expert that is slightly better early gets more flows, so it improves more, so the gate trusts it more: rich get richer. The other experts receive almost no gradient and stay untrained, so the "mixture" is one model. Fix: a load-balancing penalty on the gate's usage, noise or top-$k$ routing with capacity limits, and checking whether the data really need several regimes.
**(b)** The signal is the *hidden part of each flow* (self-supervised: the target is built from the data, with no human label). It helps because predicting fields forces a representation that captures the structure of traffic, so 2,000 labels only have to learn a simple map on top. It hurts when what the pretext task keeps is not what separates attacks: rare attack details that are useless for reconstructing typical traffic can be discarded, or the pretraining network differs from the deployment network. Using test-period inputs without labels is not label leakage, but the representation has seen the test distribution, which a real deployment would not have in advance (a transductive advantage). For an honest estimate, pretrain only on data available before the test period.
**(c)** Soft voting averages probabilities, so an overconfident model dominates: LR 0.30 with booster 0.99 averages to 0.645, an alert. The averaged "probabilities" are miscalibrated too. Fix: calibrate the booster on held-out data before voting, use hard voting, or stack with a meta-model trained on *out-of-fold* predictions so the weights are learned.
**(d)** Test-set reuse: the second test score is no longer a test. The features were chosen *after* seeing test errors, so the test set has been used for model selection, and it is optimistic exactly on the attack type that was fixed. Fix: do the per-type analysis on validation data, or report the new score as validation and evaluate once on a fresh test set (for example a later period).
**(e)** The reward defines what is learned (reinforcement learning: evaluative feedback, and the agent changes its own data). "Fewer alerts" is achieved more cheaply by blinding the sensor than by stopping attacks: reward hacking, a misspecified signal. Fix: reward the real objective (attacks contained, from independent ground truth, minus business disruption), forbid actions on the monitoring infrastructure, and audit the policy on simulated attacks.

**Course:** Essential ch.1 paradigms and learning signals; ch.5 stacking, mixture of experts, voting; ch.3 honest evaluation.

**Tempting wrong answer.** For (b): "no labels were used, so nothing leaked". True for labels, but the test inputs shaped the representation.